JWT Security Guide

JWT structure, Spring Security, Gateway boundaries, and security testing

Purpose: This document is a sample knowledge source for the DevPilot AI RAG ingestion pipeline. It is written with headings and concise sections so the service can extract text, create chunks, embed them, and retrieve relevant passages.

1. Authentication and Authorization

Authentication establishes who the caller is. Authorization decides what that authenticated caller is allowed to do. A valid login does not automatically grant access to every resource.

HTTP is stateless by default. Token-based authentication lets a client present a signed token on later requests so the server can validate identity and claims.

2. JWT Structure

A JSON Web Token commonly has three Base64URL-encoded segments separated by dots: header, payload, and signature. The header identifies token metadata and signing algorithm; the payload contains claims; the signature protects integrity.

JWT payloads are usually encoded, not encrypted. Do not place passwords, private keys, or confidential personal data in ordinary signed JWT claims.

3. Claims and Expiration

Common registered claims include sub (subject), iss (issuer), aud (audience), exp (expiration), iat (issued-at), and nbf (not-before). Applications should validate the claims relevant to their trust model.

Keep access tokens short-lived. Expiration limits how long a stolen token can be used, but it does not make token theft harmless.

4. Login and Request Flow

- Client sends credentials over HTTPS to the login endpoint.
- Server looks up the user and verifies the supplied password against its BCrypt hash.
- On success, the server issues a signed access token with an appropriate subject and expiration.
- Client sends the token as Authorization: Bearer to protected APIs.
- A trusted component validates the signature and claims before allowing the request.
- Authorization checks ensure the user can access the requested resource.

5. Spring Security Design Considerations

Use a stateless security configuration for a bearer-token API when that matches the application design. Disable form login and HTTP Basic if they are not used. Configure CSRF based on the chosen credential transport; cookie-based authentication requires careful CSRF protection.

Public endpoints such as registration, login, and health checks should be explicitly listed. Protected routes should require authentication and apply authorization rules.

BCrypt is a password hashing function, not encryption. Hashes are designed to be verified, not decrypted.

6. API Gateway and Microservices

An API Gateway can centralize routing, cross-origin policy, rate limiting, and token validation. Downstream services must still have a defined trust boundary. If services accept identity headers from the gateway, prevent direct

untrusted access and strip client-supplied versions of those headers.

Service discovery such as Eureka helps services find one another; it does not itself authenticate a request or validate a JWT.

7. Logout, Revocation, and Storage

With self-contained access tokens, logout on the client can remove the local token, but it does not automatically invalidate a copied token at the server. Stronger immediate revocation can require a denylist, token versioning, or a refresh-token strategy.

Secure, HttpOnly, SameSite cookies can reduce exposure to JavaScript token theft but require deliberate CSRF and cookie configuration. Browser storage choices have trade-offs; document the threat model and never store passwords.

8. Security Test Checklist

- Missing, malformed, expired, or incorrectly signed tokens are rejected.
- Public login and registration routes remain reachable.
- Authenticated users cannot access another user's private records.
- Untrusted clients cannot spoof identity headers.
- CORS allows only the intended frontend origins.
- Logs and error responses do not reveal tokens or secrets.

9. RAG Questions

- What is the difference between authentication and authorization?
- Does a signed JWT encrypt its payload?
- Why is BCrypt hashing rather than encryption?
- What is the limitation of client-side logout for self-contained JWTs?
- Why does Eureka not replace authentication?

End of document. Use this file for development and testing; verify all examples against your actual application configuration before treating them as production standards.